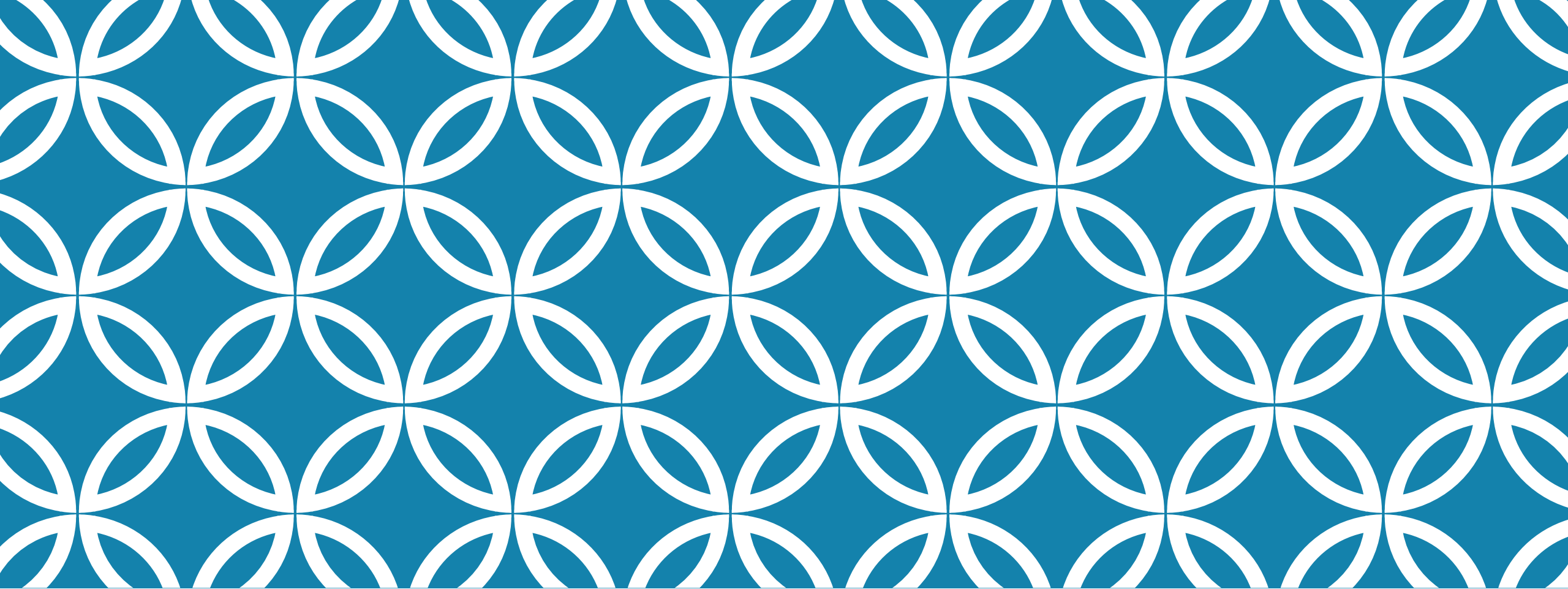
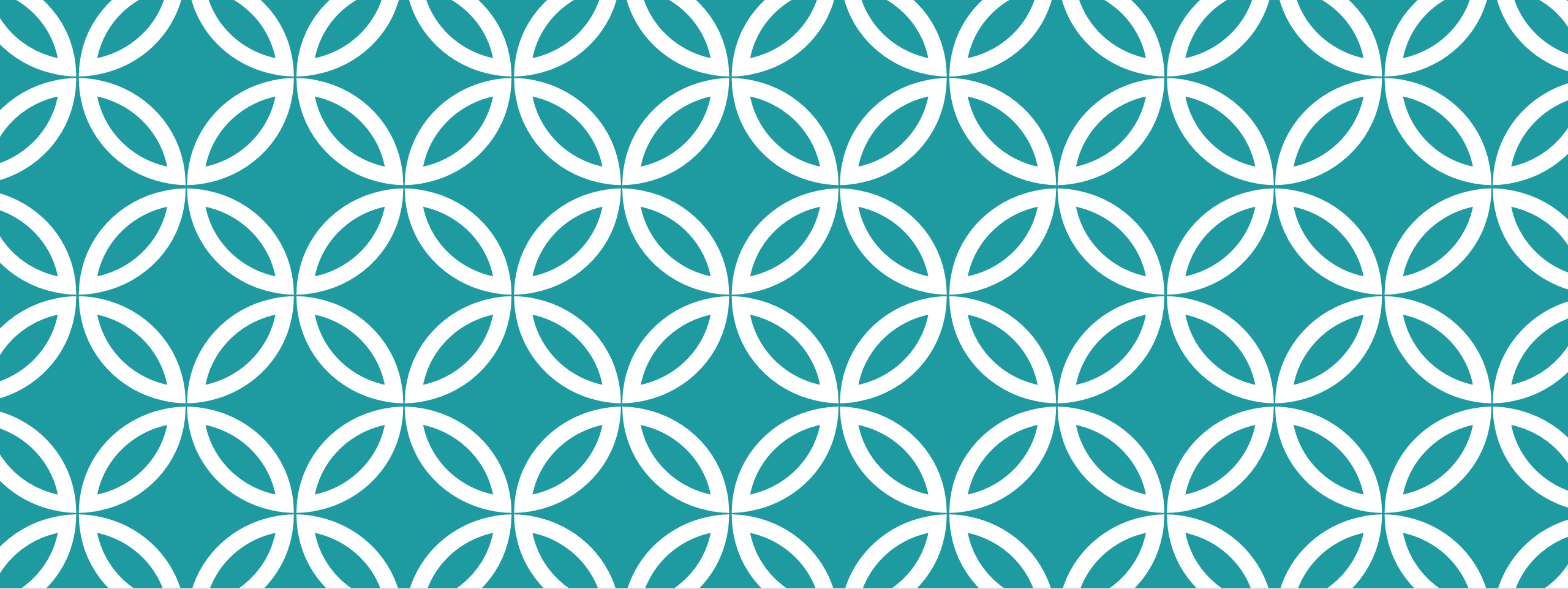




OOP - S4 SYSTEM

dr Maria Kubara, WNE UW



S4

2nd approach to OOP

S4

- ❖ S4 is a more complex system than S3
- ❖ It addresses the main issues with S3 – such as lack of the structure and problems with object validity
- ❖ It is built in R like S3 (no additional package needed)
- ❖ Uses specialized functions for creating classes `setClass()`, generics `setGeneric()`, and methods `setMethod()`
- ❖ Allows for multiple inheritance (i.e. a class can have multiple parents) and multiple object methods – method can use two or more objects of the same class (multiple dispatch)

CLASS DEFINITION IN S4

Classes in S4 work similarly like in S3, but they require more structure!

Classes in S4 are created with `setClass()` function, where we need to specify class attributes (slots) and their types.

New objects of specific class are created with `new()` function which takes as arguments the class of the object and the values of its slots.

New class is created here

```
> setClass("client4",  
+         slots = list(fname = "character",  
+                       lname = "character",  
+                       age = "numeric",  
+                       gender = "character",  
+                       married = "logical"))  
> k4 <- new(Class = "client4", # Capital letter  
+          fname = "John",  
+          lname = "Smith",  
+          age = 35,  
+          gender = "M",  
+          married = TRUE)
```

Using `new()` we can create the first object of class "client4"

CLASS DEFINITION IN S4 WITH VALIDITY!

Save class definition
to a variable

It is convenient to have a constructor function with the same name as the class. Additionally, we would like to check the structure.

Constructor can be named the same way as the class (we can omit the Class argument in new() function) if class definition will be saved to a variable of the same name as our class.

Argument validity allows for defining a checker function which ensures well defined values of slots.

```
> client4 <- setClass("client4",
+                      slots = list(fname = "character",
+                                   lname = "character",
+                                   age = "numeric",
+                                   gender = "character",
+                                   married = "logical"),
+                      validity = function(object)
+                      {
+                        if(object@age < 0) return("Age cannot be negative!")
+                        if(object@age > 100) return("Age is too large!")
+                        if(!object@gender %in% c("F", "M"))
+                          return('Incorrect gender - use "F" or "M"!')
+                        return(TRUE)
+                      })
>
> # now instead of the new() function we can use the client4()
> # function to create new objects, although of it directly
> # refers to the new() function - is just a kind of mask.
>
> client4
class generator function for class "client4" from package '.GlobalEnv'
function (...)
new("client4", ...)
```

Validity allows to
control the structure
of the object. It's a
function returning
TRUE/FALSE

Now "client4" serves as a tailored
mask for new("client4", ...) function

VALIDATION IN S4

Constructor function in S4 can include validation (as in the previous slide).

An object won't be created if the structure is invalid.

However, if we try to change the slot values later, only a warning will occur. We can check the validity of the object (if it complies with the defined structure with function `validObject()`).

```
> k4 <- new(Class = "client4", # Capital letter here!!  
+          fname = "John",  
+          lname = "Smith",  
+          age = 35,  
+          gender = "M",  
+          married = TRUE)  
>
```

```
> k4@age <- 140  
>
```

Although validity check is in place, we can change the slot value without restriction

```
> validObject(k4)
```

```
Error in validObject(k4) :  
  invalid class "client4" object: Age is too large!
```

```
>  
> # lets correct a "mistake"
```

```
> k4@age <- 40  
>
```

`validObject()` runs the "validity" function in S4 and checks if the structure is still ok

```
> validObject(k4)
```

```
[1] TRUE
```

```
> slot(k4, "age") <- "40"
```

Automatic checking works only for slot types

```
Error in checkSlotAssignment(object, name, value) :  
  assignment of an object of class "character" is not valid
```

METHODS IN S4

Methods in S4 work the same as in S3. They belong to generic functions.

First a generic function must exist (create with `setGeneric()` function), and then we can write its specific behavior which will work for our new class (`setMethod()` function).

```
> setGeneric(name = "age4",
+           def = function(x) {
+             standardGeneric("age4")
+           })
[1] "age4"
>
> age4
nonstandardGenericFunction for "age4" defined from package ".GlobalEnv"

function (x)
{
  standardGeneric("age4")
}
<environment: 0x0000022728267a98>
Methods may be defined for arguments: x
Use showMethods(age4) for currently available ones.
>
> showMethods(age4)
Function: age4 (package .GlobalEnv)
<No methods>

> setMethod("age4",
+           signature = "client4",
+           definition = function(x) x@age
+ )
>
> showMethods(age4)
Function: age4 (package .GlobalEnv)
x="client4"
```

`standardGeneric()` in S4 works the same as `UseMethod()` in S3 and is the heart of any generic function

Define new method with `setMethod()`

MULTIOBJECT METHODS IN S4

In S4 we can create methods which work for more than one object of given class.

That's one of the main advantages of S4 compared to S3.

```
# generic function
setGeneric("compare",
  # lets define two arguments
  # (objects for comparison)
  function(x, y) standardGeneric("compare"))

setMethod("compare",
  # here classes of required input objects are defined
  signature = c("client4", "client4"),
  definition = function(x, y) {
    cat(x@fname, x@lname, "is",
      ifelse(x@age == y@age,
        "as old as",
        ifelse(x@age > y@age,
          "older than",
          "younger than")),
      y@fname, y@lname, "\n")
  })
```

If a function is to work with more objects,
we need to specify more arguments (x,y)

Double signature here
– for both arguments

INHERITANCE IN S4

- ❖ Inheritance in S4 is done within the class definition
- ❖ We use argument `contains` to define the parent class from which slots will be inherited
- ❖ Similarly, parent's methods will be used if no behavior for child class is defined

```
clientPremium <- setClass("clientPremium",  
  
  # contains allows for inheriting slots from other classes  
  contains = "client4",  
  
  # new slots (unique to our new class) can be defined in typical slots  
  slots = list(  
    premium = "logical"  
  ),  
  
  # prototype allows for defining the default slot values  
  prototype = list(  
    premium = TRUE  
  ))
```

Contains allows for specifying inheritance

Helpful argument
prototype in class
definition – you can
define default slot
values here